ai-daw-tools

> An open, transparent, agent-addressable workshop for making music with LLMs. Where commercial AI music products generate finished tracks behind a black-box "go" button, this is the inverse — a system of small composable instruments where every decision is visible, every primitive is editable, and the act of building the tool is itself a musical act.

What this is

ai-daw-tools is a browser-native music environment built around three ideas:

1. **Typed musical documents under one shared context.** Every instrument reads and writes explicit JSON documents instead of opaque audio blobs. Sample tools emit `Pattern`, synth and sample-informed hybrid tools emit `SynthScene`, and effects declare `audio-stream` workflows. The common layer is `GlobalMusicContext` — BPM, swing, key, and scale — so tools stay loosely coupled like a DAW while every artifact remains serializable, inspectable, hand-editable, and LLM-mutable.

2. **Levels of agency, not levels of automation.**

   - **L1** is a tool that makes music — a sampler, a synth, a drum machine.
   - **L2** is an agent that builds new L1 tools from a natural-language description.
   - **L3** plans and builds new L2 builders — a synth-builder, an effect-builder, a microtonal-builder — each templating tools for a different domain. The first `_l3-builder` planning surface is live at `/agents/build`, and its five specialized L2 builder routes are live under `/build/<domain>`.

   The whole stack is observable: every AI call streams its prompt, reasoning, and structured decisions. Nothing happens behind a curtain.

3. **A music-theory-aware shared context.** A `GlobalMusicContext` carries the current BPM, swing, key, and scale across every tool in a session. Scales are first-class objects supporting four tuning systems (`12tet`, `EDO`, `cents`, `ratios`), so the system handles everything from standard 12-tone music to Bohlen-Pierce, just intonation, and Scala-compatible imports.

The whole thing is open source, Vercel-deployable, and bring-your-own-model. The Vercel AI Gateway abstraction means you can swap Anthropic Claude for OpenAI, Gemini, or any other provider with one config change.

Why this exists (anti-Suno, in earnest)

Suno-style "prompt → complete song" generators are fast food for music. They produce listenable artifacts efficiently and they skip every step where craft used to live. You have no idea why the result sounds the way it does, who decided what, what alternatives were suppressed, or how to push further.

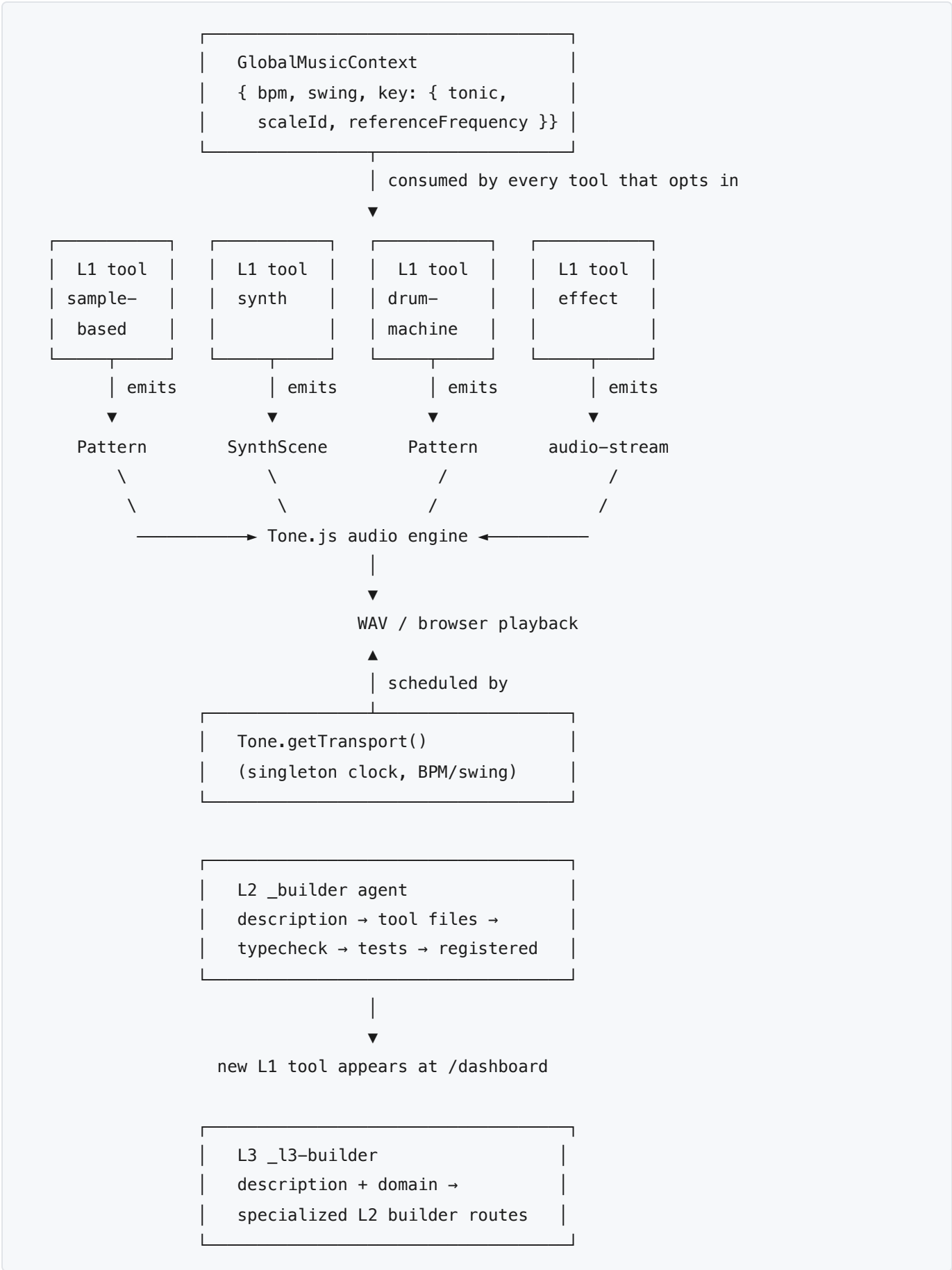
This project takes the opposite stance:

What Suno hides	What ai-daw-tools surfaces
The decision to pick a key, a tempo, a groove	A `GlobalMusicContext` you read and write
The model's reasoning	A streaming reasoning panel beside every generation
Which patterns were considered but rejected	A first-class `alternative` chunk type in the stream
What the tool can and can't do	An `AgentManifest` declaring inputs, outputs, capabilities, autonomy
When the model can't do something cleanly	An explicit `breach` chunk you can see and act on
The boundary between user intent and model output	Steps carry `metadata.createdBy: 'manual' | 'ai' | 'import'`
What "interesting music" even means	You decide. The tools are primitives.

The goal isn't to *replace* generative AI; it's to **expose** it. A pattern with `probability: 0.7` and `microShift: -0.04` isn't a black box — it's a document you can read, share, mutate, and reason about. An LLM emitting one is doing something you can supervise.

This is also a practical bet: a system this *primitive-rich* would be intractable to hand-code traditionally, but with LLMs as a primary collaborator, the cost of adding a new instrument or a new scale family or a new step parameter drops to "describe it and watch it appear." That's the L2 builder's whole job.

The system, at a glance


GlobalMusicContext
{ bpm, swing, key: { tonic,
scaleId, referenceFrequency }}
consumed by every tool that opts in
L1 tool sample-based
L1 tool synth
L1 tool drum-machine
L1 tool effect
emits
emits
emits
emits
Pattern
SynthScene
Pattern
audio-stream
Tone.js audio engine
WAV / browser playback
scheduled by
Tone.getTransport()
(singleton clock, BPM/swing)
L2 _builder agent
description → tool files →
typecheck → tests → registered
new L1 tool appears at /dashboard
L3 _l3-builder
description + domain →
specialized L2 builder routes

Everything is typed end-to-end with Zod. Every primitive is auditable. Every AI call goes through Vercel AI Gateway, so model choice is configuration.

Core concepts

The universal music document

There isn't *one* universal document type — there are **two cooperating ones** that share a tonal/transport context:

`Pattern` (lib/pattern/schema.ts)

The richer-than-MIDI step-sequencer document. Every track is an array of steps, and every step carries:

Field	Meaning
`active`	Will this step fire?
`velocity`	0..1 loudness
`microShift`	-0.5..0.5 fraction of a step — humanization beyond global swing
`probability`	0..1 chance the step actually fires this cycle
`conditions`	`everyN`, `firstOfN`, `notFirstOfN` — gates by bar index, for fills
`pitchSemitones`	Integer pitch shift (12TET)
`pitchCents`	Explicit cent offset for microtonal sample playback
`tuningRef`	`{ scaleId, degree }` reference resolved against the global scale
`playbackRate`	Direct playback-rate override
`decay`	Gate-length multiplier
`reverse`	Play the sample backwards
`chokeGroup`	Mutual-exclusion key — when one step in the group fires, others mute
`repeats`	Sub-step repeats (rolls/flams)
`sampleId` / `slot`	Which sample this step plays

A `Pattern` carries `bpm`, `swing`, `bars`, `stepsPerBar`, a list of `Track`s, and a `metadata` object including `createdBy: 'manual' | 'ai' | 'import'`, a free-form `rationale` string explaining the agency behind the pattern, and `tags`.

This document is what LLMs emit when they "generate a pattern." It's a JSON object validated by a Zod schema — there's no opaque tensor, no diffusion model.

`SynthScene` (app/tools/evolving-fm-synth/lib/schema.ts)

The synth-shaped document, used by tools that synthesize sound rather than play samples. Carries `voices` (each with `patch`, `steps`, `gain`, `pan`, `role`), `effects` (filter / delay / reverb / chorus / drive), and `macros` (`evolution`, `mutationDepth`, `brightness`, `dubSpace`, `density`, `analogDrift`).

Why a different schema? Synths have parameters samples don't (oscillator partials, modulation indices, ADSR envelopes per voice). Forcing them into a step-grid shape would lose meaning. They share `bpm`, `swing`, and `key` via the global context (see below), so they still compose with sample tools — just via adapters that translate document → document, not via a single bloated schema.

Why two schemas instead of one universal type

The earlier architectural plan considered unifying everything into one `MusicalDocument` (union, base interface, or event log). The chosen design is better: **keep the documents shape-specific, share the context.** This is how DAWs and live performers actually think — shared BPM/key, independent instruments. Cross-document composition uses small adapters (`Pattern → SynthScene` extracts MIDI from active steps; `SynthScene → audio-stream` records via `Tone.Recorder`).

The Global Music Context

```
type GlobalMusicContext = {
  bpm: number;                 // 40–260
  swing: number;               // 0–0.5
  key: {
    tonic: 'C' | 'C#' | ... | 'B';
    scaleId: string;           // references a ScaleDefinition
    referenceFrequency: number; // 400–480 Hz, A4 anchor
  };
};
```

Tools opt into pieces of this context through their manifest:

```
instrument: {
  type: 'sample' | 'synth' | 'hybrid' | 'effect' | 'builder',
  document: 'pattern' | 'synth-scene' | 'audio-stream' | 'files',
  workflow: string,            // free-form workflow tag
  usesSamples: boolean,
  usesSynthesis: boolean,
}
musicContext: {
  globalBpm: boolean,          // follow the session's BPM?
  globalKey: boolean,          // follow the session's tonic + scale?
  scaleSearch: boolean,        // expose a "find scale" affordance?
}
```

When you wire two tools into a session, both see the same key, the same tempo, and the same scale. Change the tonic from C to D and both retune simultaneously.

Scales as first-class objects

The scale system supports **four tuning paradigms**, encoded as a discriminated union (`lib/music/context.ts`):

Kind	Use it for
`12tet`	Standard Western scales as intervals over twelve-tone equal temperament
`edo`	Equal divisions of the octave: 19-EDO, 22-EDO, 31-EDO, etc. Microtonal grids.
`cents`	Arbitrary cents values per scale step. Best for ethnomusicology imports.
`ratios`	Just intonation as fraction strings: `3/2`, `5/4`, `7/4`. Bohlen-Pierce too.

Each `ScaleDefinition` carries `id`, `name`, `family`, `aliases`, `tags`, `description`, `microtonal: boolean`, and a `source` field (`curated`, `scala-compatible`, `tonal-compatible`) for interop with the Scala scale archive and the `tonal` library.

This means you can put tools into Bohlen-Pierce or 31-EDO and the global context will propagate the choice. The synth uses the scale directly, and the shared sample playback path resolves `pitchCents` / `tuningRef` into playback rates so sample tools can participate in microtonal sessions too.

Agents and the levels

Level	Role	Current state
L1	A tool that makes music	5 hand-built + 3 L2-generated, live
L2	An agent that builds new L1 tools from a description	Live base builder plus five specialized domain routes
L3	An agent that builds new L2 builders specialized per domain	Planner live at `/agents/build`; promoted routes live at `/build/sample`, `/build/synth`, `/build/effect`, `/build/hybrid`, `/build/microtonal`

Every tool — at every level — is described by an `AgentManifest` (lib/agents/contract.ts). The manifest is the contract: it declares the tool's level, its inputs and outputs, its capabilities, its `autonomy` mode (`manual` / `assist` / `driven`), its `status` (`enabled` / `disabled` / `coming-soon`), and its `origin` (`installed` / `generated`).

The dashboard at `/dashboard` is just a sorted view of every manifest in the registry. The registry is filesystem-discovered: drop a new `app/tools/<slug>/manifest.ts` and the dashboard picks it up at build time. L2-generated tools live in `.audit/generated-tools.json` and are merged in at runtime.

Exposed agency, as data

Every generation streams typed chunks back to the UI (`lib/agents/builder-contracts.ts`):

```
type BuilderStreamChunk =
  | { type: 'decision'; message: string }
  | { type: 'alternative'; message: string }
  | { type: 'breach'; message: string }
  | { type: 'tool-call'; name: string; input: unknown; result: unknown }
  | { type: 'file'; path: string }
  | { type: 'manifest'; manifest: AgentManifest }
  | { type: 'verification'; name: 'manifest' | 'typecheck' | 'tests'; passed: boolean }
  | { type: 'complete'; manifest: AgentManifest; files: string[]; registered: boolean }
  | { type: 'error'; message: string };
```

The "alternatives the agent considered but rejected" and "boundary breaches" are first-class types, not afterthoughts in a log. Reading the stream is reading the agent's process.

The same approach extends to pattern generation: every `Pattern.metadata` carries a `rationale` string that explains the decisions; AI-generated patterns mark `createdBy: 'ai'`. The chain of custody for a sound is visible.

The L2 builder, in detail

`/build` runs the level-2 agent that produces new L1 tools.

1. **You write a description** — "a tool that takes a vocal sample and stutters it like Burial."
2. **You optionally pick a reference agent** (defaults to `intelligence-sampler`, which is the canonical sample-based tool). Synth descriptions auto-route to `evolving-fm-synth`.
3. **The builder agent runs** — either `runBuilderAgent` (deterministic, scripted) or `runBuilderToolLoopAgent` (Claude-driven via `ToolLoopAgent`). The agent calls a fixed set of sandboxed tools:
   - `readSchema()` — read `lib/pattern/schema.ts`, `lib/agents/contract.ts`, `lib/samples/roles.ts`
   - `readToolFiles({ slug })` — read the reference tool's files
   - `instantiateSkeleton({ slug, name, description, instrumentType })` — render the sample Pattern or synth SynthScene template kit
   - `editToolFile({ slug, projectPath, content })` — write a file inside the sandbox
   - `runToolTypecheck({ slug })` — scoped `tsc --noEmit` against a per-tool tsconfig
   - `runToolTests({ slug })` — scoped `vitest run --project unit <slug>`
   - `validateManifest({ slug })` — re-evaluate the generated manifest, parse against schema
   - `registerTool({ slug })` — only succeeds if all three gates above pass
4. **Verification gates run on every registration attempt.** If typecheck fails, the agent gets diagnostics back as a tool result and can iterate. If sandbox boundaries are violated, a typed `SandboxBreach` is raised and the breach is surfaced in the UI rather than silently failing.

5. **The `/build` UI summarizes the run as staged gates** — brief, schema, reference, sandbox boundary, files, manifest, typecheck, tests, registry — so a producer can see whether the L2 agent is running, blocked, failed, or complete without parsing the raw NDJSON stream.
6. **The new tool appears on `/dashboard`** with `origin: generated`. You can play it, edit its pattern, share its URL, just like any hand-built tool.

The whole thing is sandboxed: the L2 agent cannot write outside `app/tools/<slug>/`, cannot install packages, cannot modify shared code in `lib/`. If it *wants* to, it surfaces a "feature request" breach — which is the hand-off signal to a human or to a future L3 agent.

Available tools (May 2026)

Slug	Type	Origin	What it does
`intelligence-sampler`	sample	installed	Canonical break-sampler. Slice a break into 8 playable slots, sequence them. The reference tool the L2 builder reads.
`grid-sampler`	sample	installed	2D grid of chops with directional traversals (`LR_serp`, `Spiral`, `Random`, etc.). Has its own tool-calling agent.
`drum-machine`	sample	installed	Multi-track sampler with polyrhythmic step counts per track (16 vs 13 vs 7) — patterns DAWs can't easily express.
`splice-lab`	sample	installed	A/B sample splice loop with `chokeGroup` mutual exclusion. The seed of multi-source composition.
`evolving-fm-synth`	synth	installed	Hybrid FM/wavetable synth with a deterministic local agent. Prompt → key/scale/BPM/macros/voices/effects/MIDI lanes. Microtonal-aware.
`_builder`	builder	installed	The L2 agent itself. Routes to `/build`.

L2-generated tools appear on `/dashboard` whenever `.audit/generated-tools.json` contains registered manifests. This checkout keeps the L2 builder live even when no generated registry file is present.

How to use this

Run a tool

1. Visit `/dashboard`.
2. Use the prompt-to-session launchpad to describe the rough track vibe. The studio agents shape the global BPM, swing, root, scale, and per-tool prompt stack in one pass.
3. Open the shared session or copy any generated tool prompt into a specific instrument.
4. Pick a sample (built-in library or upload your own).

5. Optionally type a vibe prompt and hit "Generate" — an LLM produces a typed Pattern (or SynthScene), the grid fills in live as the document streams.
6. Press play.
7. Export the Pattern/SynthScene JSON or render/record a WAV layer.

The launchpad is a canvas-first song board: song idea -> tempo -> key -> tools -> export. The first interaction is deliberately plain: describe the track, click "make stems board", and let the visible agents fill in BPM, swing, root, scale, and per-tool prompts. The primary dashboard action shapes those prompts and opens the shared session in one step, so the stored plan immediately becomes a playable track board with a small "quick stems" card for export generation. Stored session plans include the structured `GlobalMusicContext`, so reloading a session restores the actual BPM, swing, tonic, scale, and reference frequency instead of only the text prompt labels. Manual context controls, readiness checks, and the raw registry stay collapsed as advanced panels below the board. Inside a session, each selected scene slot can generate its own JSON sketch, edit the prompt in place, download that sketch, and promote it into the export/layer rack so seed, variation, and breakdown ideas can become real layer sources before the export pass. Scene headers can also generate a whole column across tools, so a seed, variation, or breakdown pass can be produced as a coordinated batch. The session workflow strip keeps scene-slot, export, and audio-layer readiness visible while a scene overlay marks active LLM generation. The layer rack can also copy or download a typed handoff manifest containing the context, scene prompts, planned layers, adapter wires, recent monitor events, and L2 builder brief. The L2 brief opens `/build` with session-derived name, slug, instrument type, and description already populated, so the builder handoff is not a generic default. The session also derives a linear arrangement sketch from the scene slots, giving the improvisational grid a song timeline before stems or JSON leave the browser. Arrangement generation can run every scene in bar order, producing all prompt scene JSONs for the song sketch without clicking each scene manually. Export-pass scene slots are promoted into the layer rack as soon as their JSON is ready, so the producer-facing source-doc checklist advances during scene and arrangement generation instead of requiring a second manual promote step. Once source docs exist, the layer rack can render every source-ready audio layer as a batch, using the same monitored Pattern/SynthScene WAV paths as the per-layer render buttons. The session workflow also exposes a one-click export-stems action that generates the export pass and renders the ready audio layers with bounded concurrency. Scene and arrangement generation also run scene slots through a bounded LLM-request queue, so one click cannot fan out every registered tool at once and future L2-generated instruments do not overload the gateway. The run quality monitor aggregates workflow status, layer readiness, recent events, gateway fallbacks, and errors into a single producer-facing health state so a session is visibly working, blocked, or ready for handoff. The handoff completion audit is stricter: it checks scene slots, source documents, audio stems, stem-bundle state, scale evidence, and runtime health, then shows a readiness score before you move stems out of the browser. The session also computes a recommended next action from the same state model, so the producer sees one clear command — generate a scene, render source-ready audio, export stems, wait for active work, or resolve a blocked run — instead of having to infer it from the board, layer rack, and monitor separately. The studio session view is producer-first by default: a responsive canvas-style board shows the one recommended action, scene prompts, workflow readiness, and a compact stem checklist without opening the dense tool/editor surfaces. Manual context controls, scale search, run-quality/runtime

monitors, completion audit, arrangement exports, artifact queue, adapter wires, manifests, and embedded tools live behind collapsible advanced panels.

Have L2 build you a new tool

1. Visit `/build`.
2. Type a description: "*a tool that takes a percussion sample and arranges it on a triangle wave with conditional fills every 4 bars.*"
3. Pick a reference agent if you want (the builder will choose one if you don't).
4. Hit "Build."
5. Watch the staged run monitor: brief → schema → reference → sandbox → files → manifest → typecheck → tests → registry. The raw stream remains below it for audit detail.
6. If the build passes, the tool now appears on `/dashboard` with `origin: generated`.
7. If the build hits a sandbox breach (e.g., "I'd need a granular-synthesis primitive that doesn't exist"), the breach is logged and you can either implement the primitive yourself or wait for the L3 builder to grow a new domain.

Plan a new L2 builder with L3

1. Visit `/agents/build`.
2. Pick a builder domain: `sample`, `synth`, `effect`, `hybrid`, or `microtonal`.
3. Type the kind of L2 builder you want.
4. Inspect the candidate L2 manifest, reference agents, template kit, sandbox root, implementation steps, and verification gates.
5. Open the matching `/build/<domain>` route to run the specialized builder with the instrument domain locked and the L3 plan visible beside the run monitor.

Build a tool by hand

Every tool is a directory under `app/tools/<slug>/` with this shape:

```
app/tools/<slug>/
├── manifest.ts               # exports a const matching AgentManifestSchema
├── page.tsx                  # RSC route shell
├── client.tsx                # 'use client' island
├── components/               # local components
├── lib/
│   ├── prompt.ts             # AI prompt builders (system + per-request)
│   └── play.ts (optional)    # Pattern → Tone.js scheduling
├── __tests__/                # vitest tests (TDD-style)
└── store.ts (optional)       # Zustand slice
```

Drop a new folder, and the filesystem-driven registry picks it up at next build. No edits to a global index, no manual dispatch wiring. Tools that don't have a custom prompt builder fall back to

`buildGenericToolPrompt` (lib/agents/dispatch.ts), which uses the manifest's description as a system-prompt seed.

Bring your own model

The whole AI integration goes through Vercel AI Gateway:

```
// lib/ai/gateway.ts
import { gateway } from '@ai-sdk/gateway';
gateway('anthropic/claude-sonnet-4.6')      // dot-format model identifier
```

To swap to another provider, change one env var (`AI_GATEWAY_MODEL`) or pass a specific model into `getGatewayModel(modelId)`. The Gateway routes to OpenAI, Anthropic, Google, Groq, and others. For deployed environments use OIDC tokens via `vercel env pull`. For local development you can use `AI_GATEWAY_API_KEY` in `.env.local`.

To add a model that the Gateway doesn't yet route, install the relevant `@ai-sdk/<provider>` package and pass that provider's model directly into the SDK's `streamText` / `Output.object` calls (see `lib/ai/pattern-generation.ts`).

Compose tools (in progress)

A `/sessions/[id]` route mounts multiple tools under the shared `GlobalMusicContext`. The prompt-to-session launchpad persists the latest studio plan, and the session page renders it as a DAW-style track board: every enabled L1 music tool gets a row with its role, document type, scene slot, export target, and open/copy actions. Export scene slots feed a session artifact queue, which turns planned pattern/synth documents into concrete JSON and WAV layer targets. Pattern-tool artifacts call the existing generation endpoint from the session and download Pattern JSON or rendered WAV layers. Synth-scene artifacts call `/api/synth`, cache the returned SynthScene, and download either SynthScene JSON or an offline-rendered synth WAV layer. Audio-stream effect tools enter the same board as stream artifacts, which keeps send/insert effect plans visible even when they do not produce a Pattern or SynthScene source document. A separate session layer rack opens as a compact stem checklist grouped by tool, with source JSON and audio-stem readiness side by side. Detailed artifact cards, manifest copy/download actions, evidence, errors, export filenames, and rerun controls stay one disclosure deeper. For performance, the session embeds one active tool at a time instead of mounting every audio surface simultaneously. A separate `/perform/[id]` surface reuses the same session engine but hides the dense edit panels and embedded tool frame, leaving the workflow strip, monitor, track board, and layer rack as a stage/export view. Studio and perform modes both keep the run quality monitor visible, so gateway fallbacks, source-doc gaps, and render errors remain obvious even when the dense editor is hidden. The session also exposes visible adapter wires that turn one tool's export artifact into a concrete prompt for another compatible tool, including pattern→pattern, pattern→synth-scene, and synth-scene→pattern routing.

This makes it possible to:

- Mount the FM synth + the break sampler in one page under shared key/tempo
- Turn the grid-sampler's output pattern into a drum-machine adapter prompt
- Hit "find scale" and let the scale-search agent pick a key from the session prompt plus pitch evidence extracted from generated Pattern/SynthScene artifacts, rendered WAV buffers, and live output recordings posted by embedded tools
- Audition/apply scale candidates directly from the session so key selection is steerable instead of a one-shot black-box result

---

Schemas reference (the contract)

All schemas live under `lib/`. They're Zod schemas — runtime-validated, with TypeScript types inferred via `z.infer<>`. They are the source of truth.

Schema	Path	What it validates
`PatternSchema`	`lib/pattern/schema.ts`	The universal step-sequencer document
`SynthSceneSchema`	`app/tools/evolving-fm-synth/lib/schema.ts`	The synth-scene document
`AgentManifestSchema`	`lib/agents/contract.ts`	Every tool's contract
`GlobalMusicContextSchema`	`lib/music/context.ts`	Session-wide BPM, swing, key, scale
`ScaleDefinitionSchema`	`lib/music/context.ts`	A tunable scale: 12tet/edo/cents/ratios
`InstrumentMusicContextSchema`	`lib/music/context.ts`	Per-tool opt-in flags for global context
`BuilderStreamChunkSchema`	`lib/agents/builder-contracts.ts`	Streaming events from the L2 builder
`GeneratePatternRequestSchema`	`lib/ai/contracts.ts`	The shape of `/api/generate` calls

Adding new fields to `Pattern` or `SynthScene` is the lowest-cost way to introduce new musical capabilities — every tool inherits them automatically, and the L2 builder learns about them via `readSchema()`.

Architecture

Tech stack

Layer	Choice	Why
Framework	Next.js 16 (App Router)	RSC route shells + `'use client'` islands; first-class browser-API support
Language	TypeScript 6, strict	The schema contracts only work with strict types
Audio	Tone.js 15	Sample-accurate scheduling, internal lookahead-scheduler (Chris Wilson "Tale of Two Clocks"), `Tone.Offline()` for tests
AI	Vercel AI SDK v6	`streamText` + `Output.object()` for structured output, `ToolLoopAgent` for tool-calling agents, reasoning streaming
AI gateway	Vercel AI Gateway	Provider-agnostic, OIDC-authenticated, dot-format model strings
Schemas	Zod 4	Runtime validation + TS inference; the contract between tools, agents, and AI
State	Zustand 5	Per-tool UI state; audio scheduling stays out of React
UI	Tailwind 4 + shadcn/ui + Geist Mono	Technical/minimal aesthetic — the concepts are the visible content, not the chrome
Testing	Vitest 4 + Playwright	Multi-project: `unit` (happy-dom) + `audio` (browser/chromium for `Tone.Offline`)
Property tests	fast-check	For pattern transforms and schema round-trips
API stubs	MSW	For deterministic AI-related tests

Folder layout

```
ai-daw-tools/
├── app/
│   ├── api/                    # Route handlers
│   │   ├── generate/           # Pattern generation (per-tool dispatch)
│   │   ├── build/              # L2 tool generation
│   │   ├── synth/              # SynthScene generation
│   │   ├── music/scale-search # Scale-search agent
│   │   └── suggestions/        # Prompt suggestions
│   ├── dashboard/              # Tool registry view
│   ├── build/                  # L2 builder UI
│   └── tools/<slug>/           # Each tool is a directory
├── lib/
│   ├── agents/                 # AgentManifest, registry, sandbox, builder-tools, templates
│   ├── audio/                  # Tone.js wrappers, pattern→audio, BPM detection, WAV render
│   ├── ai/                     # Gateway, generation, tool-calling agents, contracts
│   ├── pattern/                # PatternSchema, defaults, URL codec
│   ├── music/                  # GlobalMusicContext, scale catalog, scale-agent, React hooks
│   └── samples/                # Sample library, IndexedDB upload, resolver
├── components/                 # Shared UI: AudioBootstrap, SamplePicker, shadcn primitives
├── public/samples/             # Bundled CC0 samples
└── .audit/
    ├── generated-tools.json    # L2-generated manifest registry (committed)
    └── typecheck/              # Per-tool tsconfigs emitted by the builder
```

How a pattern reaches your speakers

1. A tool collects a `Pattern` (manual edits, AI generation, URL hydration).
2. `lib/audio/pattern.ts` walks the pattern into a scheduled event plan: checks `active` / `probability` / `conditions`, applies `microShift`, resolves `sampleId` / `slot`, and hands the events to the shared sampler playback path. `lib/audio/sample-engine.ts` resolves the shared sample segment first: slice boundaries snap toward nearby zero crossings, reversed slices use the mirrored buffer offset, and short attack/release fades are applied to every live trigger, audition, manual stop, and offline WAV render.
3. `Tone.getTransport()` is the singleton clock; BPM/swing live there.
4. `AudioBootstrap` (`components/audio-bootstrap.tsx`) ensures `Tone.start()` runs inside a user gesture before any audio code touches the `AudioContext`.
5. `Tone.Offline()` is used in tests to render the same pattern deterministically so we can assert event sequences without real-time playback.

How an L2 generation works (full trace)

```
POST /api/build  { description, name?, slug?, referenceAgent?, register? }
   │
   ▼
app/api/build/handler.ts
   │  validates BuildToolRequestSchema
   │  calls runBuilderAgent(...) (or runBuilderToolLoopAgent for AI-driven)
   ▼
runBuilderAgent  (deterministic path)
   │  emit({ type: 'decision', message: 'using ${referenceAgent} as ref' })
   │  readSchema()                          → emit tool-call
   │  readToolFiles({ slug: referenceAgent })→ emit tool-call
   │  emit({ type: 'alternative', message: 'not modifying shared lib/...' })
   │  instantiateSkeleton({ slug, name, description })
   │     │  uses lib/agents/templates/tool-skeleton/files.ts
   │     │  renders sample Pattern or synth SynthScene manifest, page, client,
   │     │  prompt, tests
   │     │  writes each file via writeSandboxedFile → assertWithinTool
   │     └→ emit file chunks per write
   │  validateGeneratedManifest({ slug })   → emit tool-call + verification
   │  if (!register) → emit complete (unregistered)
   │  registerTool({ slug })
   │     │  validateGeneratedManifest (again, defensive)
   │     │  runToolTypecheck  → tsc -p .audit/typecheck/tsconfig.<slug>.json
   │     │  runToolTests      → vitest run --project unit app/tools/<slug>
   │     │  if any gate fails → emit verification(passed:false) + throw
   │     │  if all pass → writeGeneratedAgentManifest into .audit/generated-tools.json
   │     └→ emit verification(typecheck), verification(tests), complete
   ▼
The new tool appears at /tools/<slug> and on /dashboard.
```

A sandbox breach during any file write surfaces as a typed `breach` chunk: `{ type: 'breach', message: 'writes restricted to app/tools/<slug>/; got <bad path>' }`. The agent sees the error in its tool result and can either escalate or retry.

Tool-to-tool composition (the design)

Two compatible documents compose **directly**: a `Pattern` from tool A becomes an input track of tool B if they both declare `instrument.document: 'pattern'`. URL codec, generation engine, and audio engine all speak the same shape — no marshalling.

Cross-document composition uses **adapters**:

From	To	Adapter
`Pattern`	`SynthScene`	Extract active steps as MIDI; assign to a voice; preserve velocity/probability
`SynthScene`	`Pattern`	Project active MIDI as triggers on a sampler track; lose voice/timbre info
Either	`audio-stream`	Record via `Tone.Recorder` over N bars
`audio-stream`	`Pattern`	Onset-detect; emit triggers at detected onsets

This is what the planned `/sessions/[id]` wires UI exposes. Same-document connections are free; cross-document connections show the adapter as a small labeled node so the user knows what's being lost or transformed.

Getting started

Install

```
git clone <this-repo>
cd ai-daw-tools
pnpm install
```

Configure your AI provider

For local dev, get a Vercel AI Gateway key:

```
echo 'AI_GATEWAY_API_KEY=...' > .env.local
# optional override
echo 'AI_GATEWAY_MODEL=anthropic/claude-sonnet-4.6' >> .env.local
```

For deployed environments, prefer OIDC:

```
vercel link
vercel env pull
```

The gateway then auths automatically via `VERCEL_OIDC_TOKEN`. No long-lived keys.

Run

```
pnpm dev               # Next.js dev server
pnpm test              # Vitest, both projects
pnpm test:watch        # Watch mode
pnpm test:ui           # Vitest UI
pnpm typecheck         # tsc --noEmit
pnpm lint              # eslint
pnpm build             # production build
```

Open `http://localhost:3000`. The dashboard shows every registered tool.

Deploy

```
vercel deploy --prod
```

Or click "Deploy to Vercel" from the repo. The audit log (`.audit/generated-tools.json`) is intentionally committed, so generated tools travel with the deployment.

Roadmap

In short:

Shipped:

- V1 foundation, 4 hand-built sample tools, the FM synth
- L2 builder agent (deterministic + AI-driven paths)
- L2 builder run monitor that converts streamed builder chunks into nine visible gates — brief, schemas, reference, sandbox, files, manifest, typecheck, tests, registry — with next-action copy for running, blocked, failed, stopped, and complete states
- Generated-tool audit panel on `/build` that reads `.audit/generated-tools.json` and summarizes L2-created tools by readiness, instrument type, document type, prompt/context/output coverage, and registry issues
- L2 template support for all four current generated instrument domains: sample `Pattern` tools, synth `SynthScene` tools, effect `audio-stream` tools, and sample-informed hybrid `SynthScene` tools
- Generated-tool registry support for L2-built tools
- Global music context with microtonal scale support
- Scale-search AI agent and curated `ScaleDefinition` catalog
- Filesystem-driven registry, scoped per-tool typecheck, OIDC auth, multi-project Vitest
- Canvas-first dashboard song board with five plain nodes: song idea, tempo, key, tools, and export; manual context controls, readiness monitoring, and the raw tool registry stay collapsed as advanced panels

- Prompt-to-session dashboard launchpad with tempo, scale, per-tool prompt planning, separated BPM/swing and key/vibe prompt stages, and an agent activity monitor; the primary action shapes prompts and opens the shared session directly with a stem-generation handoff intent
- Shared `StudioActivityEvent` monitoring model for launchpad and session events (`tempo-suggest`, `scale-search`, `tool-plan`, prompt copy, scene focus)
- Manifest-driven session track board with persisted scene slots and one active embedded tool for browser performance
- Session board scene columns (`seed`, `variation A`, `breakdown`, `export`) that turn each tool prompt into launchable scene slots with in-grid runtime badges for generated/ready/error states
- Scene-level generation buttons that batch-generate a whole seed/variation/ breakdown/export column across registered tools while preserving per-clip monitor events and board status badges; export-pass scene generation auto-promotes ready Pattern/SynthScene source docs into the layer rack
- Session workflow strip that summarizes shared context, scene-slot readiness, export readiness, audio-layer readiness, and the current scene-generation state
- Four-track mixing console with per-track volume, pan, mute, and solo controls that are applied to scene-slot audio previews
- Session run quality monitor that combines workflow state, layer errors, gateway fallback evidence, source-doc gaps, recent events, and export/audio readiness into one visible health panel in studio and perform modes
- Browser runtime monitor in studio and perform sessions that tracks frame cadence, long frames, heap pressure when available, bounded LLM/audio queue width, and embedded-tool load so DAW sessions expose browser pressure before a recording/export run
- Handoff completion audit that scores whether a session is actually ready for stem handoff by checking scene slots, source documents, audio stems, the stem bundle manifest, scale evidence, logical session health, and browser runtime pressure
- Session recommended-next-action model that turns workflow state, layer readiness, source-ready audio, active renders, and errors into one primary producer action for studio and perform modes
- Progressive-disclosure session layout: a responsive canvas-style session board comes before a compact per-tool stem checklist; manual context, scale search, raw logs, run-quality/runtime monitors, completion audit, arrangement/artifact details, adapter wires, manifests, and embedded tools are available as advanced panels
- Board-level generation overlay for scene batches, so session-wide LLM work is visually obvious instead of hidden behind per-slot state
- Selected scene slot editing and generation: seed/variation/breakdown slots can be manually rewritten, generate Pattern/SynthScene JSON, download it, and audition ready Pattern/SynthScene audio before promoting the result into the export layer rack as a source document
- Session artifact queue that maps export scene slots to planned JSON/WAV outputs and logs artifact focus/copy actions in the monitor
- Session layer rack that turns planned outputs into a producer-facing per-tool stem checklist with source JSON and audio-stem status first, while detailed artifact controls and manifests stay behind a secondary disclosure

- Session handoff manifest export: one JSON document packages the shared musical context, scene prompts, layer checklist, adapter wires, recent monitor events, and L2 builder brief for downstream agents or a producer handoff
- Session stem-bundle manifest export: a DAW-oriented JSON handoff that lists source documents, audio stems, ready files, missing render prerequisites, blocked errors, import order, BPM, swing, tonic, scale, and reference frequency
- Arrangement sketch export: session scenes become a linear bar timeline with section starts, durations, scene references, and copy/download JSON actions
- Arrangement-level generation: run all scene columns in timeline order so a full set of seed/variation/breakdown/export scene slots can be generated as one monitored operation
- Session-aware L2 builder seed URLs: the prompt-to-session launchpad derives a coherent builder name, slug, instrument type, and description when opening `/build` from the L2 brief
- Session-generated pattern artifacts: pattern export rows can call `/api/generate`, monitor the LLM run, store the returned Pattern, and download JSON/WAV layer outputs without leaving the session
- Session-generated synth artifacts: synth export rows can call `/api/synth`, monitor the LLM/local-agent run, store the returned SynthScene, and download SynthScene JSON or an offline-rendered synth WAV layer from the session
- Session-generated hybrid artifacts preserve their sample-informed boundary: hybrid SynthScene seeds include source-sample context and metadata that marks the sample as timbral/rhythmic prompt evidence, not fake resynthesis
- Session-generated effect artifacts: audio-stream effect scene slots now produce bounded Effect patch JSON, can be promoted into the layer rack as source-ready documents, and keep live-recorded effect audio as an explicit tool-level capture step
- Source-ready audio batch render: once export-pass Pattern/SynthScene source docs are ready, the session layer rack can render every pending WAV layer in one monitored action while preserving the same per-artifact evidence trail
- One-click export-stem workflow: the session workflow strip can generate the export-pass source docs and render their WAV layers as a bounded-concurrency monitored bundle, reducing the prompt-to-stem path to one action
- Performance/show route at `/perform/[id]`: a stripped session view that keeps the monitored workflow, scene board, and layer rack while hiding global edit modules, adapter-wiring panels, and embedded tool iframes
- Session wires panel with visible pattern↔pattern, pattern→synth-scene, and synth-scene→pattern adapter prompts for routing one export into another tool
- Artifact/audio-aware session scale search: generated Pattern/SynthScene artifacts and rendered WAV buffers contribute pitch-class evidence to the scale-search prompt, and the session UI shows the evidence before retuning the shared context
- Session scale-loop monitor that turns artifact, rendered-audio, and live-output pitch evidence into typed visible sources with source counts, pitch-class labels, candidate counts, and next-action copy before retuning the session
- One-bar session scale capture: the session scale panel can record a BPM-sized bar from the shared output bus, ingest it as typed `bar-capture` evidence, and feed the captured pitch classes into the scale-search prompt and monitor

- Session-generated ad-hoc scales: the scale panel can register an evidence-based `session-generated` `ScaleDefinition`, apply it through the same global context path as curated scales, and expose it to synth/sample consumers through the shared scale catalog
- Embedded tools broadcast live output recording evidence back to the session, so recorder captures from sample tools and synth tools can retune the shared scale search without a separate upload step
- Session scale-search results expose selected and alternative key/scale candidates with browser-side scale auditions and one-click apply controls
- Scene slots hydrate tool prompt fields via URL state, so opening an embedded tool from a session lands directly on the selected scene prompt
- Registry-backed studio readiness monitor covering L1 tools, the four sample workflows, global context opt-in, prompt surfaces, L2 builder availability, and the currently supported L2 sample/synth/effect/hybrid skeleton domains
- Shared sampler declick path for all sample tools and generated sample-tool templates: zero-crossing slice smoothing, per-trigger fades, graceful stop release, and click-safe offline WAV exports
- Shared sample playback tuning path: `pitchCents` and `tuningRef` resolve into microtonal playback rates against the active `GlobalMusicContext`
- Live playback agency for all four sample tools: the shared trace bus emits fired/skipped/probability/condition/microShift events, the transport shows a live trace, and every sample grid overlays recent step decisions in place
- L2 sample-tool templates now inherit pro controls: abortable generation, prompt URL hydration, WAV render, JSON copy, recording, and live trace
- L2 synth-tool templates now generate `SynthScene` instruments with global BPM/key/scale sync, LLM generation overlays, WAV render, JSON copy, recording, and gateway/local-agent fallback behavior
- L2 effect-tool templates now generate `audio-stream` instruments with prompt shaped live-input effect patches, bounded filter/drive/delay controls, generation overlays, patch JSON copy, and record-output support
- L2 hybrid-tool templates now generate sample-informed `SynthScene` instruments with a source-sample picker, global BPM/key/scale sync, LLM generation overlays, WAV render, JSON copy, recording, and explicit UI copy that marks the boundary: the sample is prompt context, not fake resynthesis
- L3 `_l3-builder` planning surface at `/agents/build`: domain-specific plans for `_sample-builder`, `_synth-builder`, `_effect-builder`, `_hybrid-builder`, and `_microtonal-builder`, including candidate manifests, references, template kits, sandbox roots, handoff prompts, and verification gates
- Specialized L2 builder routes promoted from those plans: `/build/sample`, `/build/synth`, `/build/effect`, `/build/hybrid`, and `/build/microtonal` reuse the verified builder engine with locked instrument domains, domain-specific references, and visible L3 verification gates

Next, in order:

1. **Deepen the specialized builders**. Split their template kits and prompts further where the shared builder engine is too generic, starting with richer synth and microtonal sample defaults.

Plus a longer-horizon set of optional tracks: audio-aware AI embeddings, live regeneration, WebMIDI output, audit-log review UI.

Contributing

This is open source and bring-your-own-model. Contributions especially welcome for:

- **New scales** — add a `ScaleDefinition` to `lib/music/scale-catalog.ts`. Microtonal, ethnomusicological, and mathematical scales are first-class.
- **New L1 tools** — drop a directory under `app/tools/<slug>/`; the registry picks it up. Use `intelligence-sampler` for sample tools or `evolving-fm-synth` for synth-scene tools as references.
- **New adapters** — connect documents that don't yet talk to each other.
- **New AI providers** — extend `lib/ai/gateway.ts` or add a provider-specific module under `lib/ai/`.
- **Specialized L2 builders** — deepen `_synth-builder`, `_effect-builder`, `_hybrid-builder`, or `_microtonal-builder` when a domain needs a sharper template kit than the shared builder engine. The codebase is TDD-shaped. Every primitive ships with colocated tests: `pnpm test` should be green before any PR.

Project conventions

- Tools use `'use client'` islands inside RSC route shells. Audio code lives inside the client boundary; AI code lives in `'server-only'` modules.
- Manifests are validated at registry-load time. Invalid manifests log clearly and the tool is skipped (not a hard failure).
- L2-generated tools must pass `tsc --noEmit -p tsconfig.<slug>.json` and `vitest run --project unit app/tools/<slug>` before being registered.
- All AI calls go through `getGatewayModel()`. Don't construct provider clients directly — that breaks the swap-the-provider promise.
- Tests stub the AI provider via MSW. No live model calls in CI.

Influences and prior art

This system is in conversation with a long lineage:

- **Tone.js**, Chris Wilson's "A Tale of Two Clocks," Mohayonao's Web Audio repos, the Web Audio API spec — the JS audio foundation.

- **Miller Puckette's *Theory and Techniques of Electronic Music*** — the Max/MSP and Pure Data approach to patcher-style composition. The "tools that build tools" framing is a Puckette inheritance.
- **Gordon Reid's *Synth Secrets*** (Sound on Sound, 1999–2007) — the synthesis literacy the FM synth's prompt builder leans on.
- **The Scala scale archive** and **the `tonal` library** — interop targets for the microtonal subsystem.
- **AudioCraft (MusicGen, AudioGen, EnCodec, MAGNeT, JASCO)** — the "agent environment for music" pitch from Meta AI Research that frames the multi-LLM orchestration possibilities.
- **"Interfaces after AI" — Tessitura** (Orpheus Instituut) — the working-register metaphor that informs the "exposed agency" thesis.

And in counterpoint:

- **Suno**, **Udio**, **MusicGen as a black-box product** — what this is trying *not* to be.

The system is a workshop, not a vending machine. The hope is that a musician (or anyone curious) can read a generated pattern, understand exactly why it sounds the way it does, edit it, share it, and push it toward sounds that don't yet exist in anyone's training set.